

Esame Programmazione di Sistema / API Programming

Data: 2023-07-07

QUESTIONS

Domanda 1

Si spieghi il concetto di possesso in relazione agli smart pointers. Come viene gestito il ciclo delle risorse quando si utilizzano smart pointers?

Infine, si espliciti il ciclo di vita delle risorse nel seguente esempio:

```
{  
    let mut i = 10;  
    let bi1 = Box::new(i);  
    let mut bi2 = Box::new(*bi1);  
    *bi2 = 20;  
    i = *bi2;  
    println!("{}", i, bi1, bi2);  
}
```

Domanda 2

Si descriva la gestione della memoria in Rust e si spieghi come vengono evitati i problemi di sicurezza comuni come le violazioni di accesso o la presenza di puntatori nulli.

Domanda 3

Si illustri come sia possibile gestire correttamente le situazioni di errore in Rust, distinguendo tra `Option` e `Result`.

Domanda 4

Una `DelayedQueue<T: Send>` è un particolare tipo di coda non limitata che offre tre metodi principali, oltre alla funzione costruttrice:

1. `offer(&self, t: T, i: Instant)`: Inserisce un elemento che non potrà essere estratto prima dell'istante di scadenza `i`.

2. `take(&self) -> Option<T>`: Cerca l'elemento `t` con scadenza più ravvicinata: se tale scadenza è già stata oltrepassata, restituisce `Some(t)`; se la scadenza non è ancora stata superata, attende senza consumare cicli di CPU, che tale tempo trascorra, per poi restituire `Some(t)`; se non è presente nessun elemento in coda, restituisce `None`. Se, durante l'attesa, avviene un cambiamento qualsiasi al contenuto della coda, ripete il procedimento suddetto con il nuovo elemento a scadenza più ravvicinata (ammesso che ci sia ancora).
3. `size(&self) -> usize`: Restituisce il numero di elementi in coda indipendentemente dal fatto che siano scaduti o meno.

Si implementi tale struttura dati nel linguaggio Rust, avendo cura di renderne il comportamento thread-safe. Si ricordi che gli oggetti di tipo `Condvar` offrono un meccanismo di attesa limitata nel tempo, offerto dai metodi `wait_timeout(...)` e `wait_timeout_while(...)`.

SOLUTIONS

Soluzione 1

Gli smart pointer in Rust, come `Box<T>`, `Rc<T>` e `Arc<T>`, implementano i trait `Deref` e `Drop`. Il concetto di ownership (possesso) si applica agli smart pointer nel senso che essi “possiedono” la memoria a cui puntano. Quando lo smart pointer esce di scope, il suo metodo `drop` viene chiamato automaticamente (seguendo il pattern RAII - *Resource Acquisition Is Initialization*) e la memoria heap sottostante viene deallocata. Per i puntatori a possesso singolo come `Box`, c'è un solo owner alla volta. Per i puntatori a possesso multiplo come `Rc` o `Arc`, viene mantenuto un conteggio dei riferimenti, e la memoria viene liberata solo quando l'ultimo owner viene distrutto.

Nel codice fornito, il ciclo di vita delle risorse è il seguente:

1. `let mut i = 10;` - `i` è un intero (tipo primitivo) allocato sullo stack.
2. `let bi1 = Box::new(i);` - Il valore di `i` viene copiato in una nuova allocazione sull'heap. `bi1` diventa l'owner di questa memoria heap, che contiene il valore 10.
3. `let mut bi2 = Box::new(*bi1);` - Il valore puntato da `bi1` (10) viene dereferenziato e copiato (poiché `i32` implementa `Copy`) in una *nuova* allocazione sull'heap. `bi2` diventa l'owner di questa nuova memoria. Ora `bi1` e `bi2` puntano a due allocazioni heap distinte, entrambe contenenti 10.
4. `*bi2 = 20;` - Il valore nella memoria heap posseduta da `bi2` viene aggiornato a 20. La memoria di `bi1` rimane inalterata.
5. `i = *bi2;` - Il valore 20 viene dereferenziato e copiato nuovamente in `i` sullo stack.
6. `println!(...);` - Viene stampato l'output: 20 10 20.
7. Alla fine del blocco `{}`, le variabili vengono distrutte in ordine inverso rispetto alla loro creazione:
 - `bi2` esce di scope: il suo `Drop` dealloca l'area di memoria heap contenente 20.
 - `bi1` esce di scope: il suo `Drop` dealloca l'area di memoria heap contenente 10.
 - `i` viene rimosso dallo stack senza bisogno di operazioni speciali, in quanto allocato sullo stack.

Soluzione 2

La gestione della memoria in Rust si basa su un sistema di ownership, borrowing (prestiti) e lifetimes (tempi di vita), valutato rigorosamente a tempo di compilazione dal *borrow checker*. A differenza di C/C++, Rust non richiede `malloc/free` manuali, né si affida a un garbage collector come Java. Utilizza invece il pattern RAII: ogni risorsa ha un “owner”, e quando questo esce di scope, la risorsa viene automaticamente deallocata.

Rust previene i problemi comuni di sicurezza della memoria attraverso le seguenti regole:

- **Dangling pointers / Use-after-free:** I lifetimes garantiscono che i riferimenti non sopravvivano mai ai dati a cui puntano. Se un dato viene deallocato, nessun riferimento ad esso potrà essere utilizzato.

- **Data races e violazioni di accesso:** Le regole del borrowing permettono di avere in un dato momento o *molteplici riferimenti immutabili* (&T) OPPURE *esattamente un riferimento mutabile* (&mut T), ma mai entrambi. Questo concetto (“aliasing XOR mutability”) previene a tempo di compilazione le data races e l’invalidazione dei riferimenti (ad es. durante la modifica concorrente di una struttura dati). Inoltre, per evitare le violazioni di accesso dovute a buffer overflow, il compilatore inserisce *boundary checks* trasparenti ogni volta che si accede ad un array/slice tramite indice.
- **Puntatori nulli (Null pointer dereference):** Rust elimina del tutto i puntatori nulli non includendo la keyword `null`. Al suo posto, il linguaggio impone l’uso dell’enum `Option<T>` (`Some(T)` o `None`). Essendo `Option<T>` un tipo distinto, il compilatore obbliga il programmatore a gestire esplicitamente il caso `None` (es. tramite `match` o `if let`) prima di poter estrarre il valore sottostante.

Soluzione 3

In Rust, gli errori vengono gestiti principalmente tramite due enumerazioni generiche, `Option<T>` e `Result<T, E>`, evitando l’uso del meccanismo delle eccezioni tipico di linguaggi come C++ o Java (che introduce overhead a runtime e nasconde il flusso di controllo).

- **`Option<T>`:** Viene utilizzato quando l’assenza di un valore è un risultato normale e atteso, piuttosto che una condizione di errore vera e propria. Ha due varianti: `Some(T)` (che contiene il valore) e `None` (che ne rappresenta l’assenza). Ad esempio, la ricerca di un elemento in un vettore restituisce `Option`, così come il `pop` da uno stack. Il chiamante è obbligato a gestire il caso `None`.
- **`Result<T, E>`:** Viene utilizzato per le operazioni che possono fallire esplicitamente ed è utile sapere *perché* sono fallite. Ha due varianti: `Ok(T)` (che rappresenta il successo e contiene il risultato) e `Err(E)` (che rappresenta il fallimento e contiene i dettagli dell’errore). Per esempio, l’apertura di un file restituisce `Result<File, std::io::Error>`.

Entrambi i tipi offrono una ricca serie di combinatori (es. `map`, `and_then`, `unwrap_or`) per concatenare operazioni in modo elegante. Inoltre, Rust fornisce l’operatore `?`, che permette una propagazione concisa degli errori. Quando `?` viene accodato a una funzione che restituisce `Result` o `Option`, questo estrarrà il valore in caso di successo, oppure ritornerà immediatamente l’errore/`None` al chiamante, riducendo drasticamente il boilerplate rispetto alla gestione puntuale tramite `match`.

Soluzione 4

```
use std::sync::{Mutex, Condvar};
use std::time::Instant;

pub struct DelayedQueue<T: Send> {
    state: Mutex<Vec<(T, Instant)>>,
    cv: Condvar,
```

```

}

impl<T: Send> DelayedQueue<T> {
    pub fn new() -> Self {
        Self {
            state: Mutex::new(Vec::new()),
            cv: Condvar::new(),
        }
    }

    pub fn offer(&self, t: T, i: Instant) {
        let mut state = self.state.lock().unwrap();
        state.push((t, i));
        // Segnaliamo la condvar in quanto un nuovo elemento e'
        stato aggiunto
        // ed e' necessario rivalutare la scadenza piu'
        ravvicinata.
        self.cv.notify_all();
    }

    pub fn size(&self) -> usize {
        let state = self.state.lock().unwrap();
        state.len()
    }

    pub fn take(&self) -> Option<T> {
        let mut state = self.state.lock().unwrap();

        loop {
            if state.is_empty() {
                return None;
            }

            // Cerchiamo l'elemento con la scadenza piu'
            ravvicinata (minimo Instant)
            let mut min_idx = 0;
            let mut min_time = state[0].1;

            for (idx, &(_, time)) in state.iter().enumerate().skip
(1) {
                if time < min_time {
                    min_time = time;
                    min_idx = idx;
                }
            }

            let now = Instant::now();
            if min_time <= now {
                // La scadenza e' gia' passata, possiamo estrarre
                e restituire l'elemento
                let (val, _) = state.remove(min_idx);

```

```

        return Some(val);
    } else {
        // La scadenza non e' ancora superata, dobbiamo
attendere.

        // Calcoliamo quanto manca alla scadenza.
        let timeout = min_time.duration_since(now);

        // wait_timeout attende fino alla scadenza o
finche' non viene notificato
        let (new_state, _timeout_res) = self.cv.
wait_timeout(state, timeout).unwrap();
        state = new_state;

        // Indipendentemente dal fatto che sia scattato il
timeout o che
        // siamo stati svegliati da una nuova offer(), il
loop riparte e
        // rivaluta qual e' il nuovo elemento con scadenza
piu' vicina
        // (rispettando il vincolo di ricalcolare se ci
sono cambiamenti).
    }
}
}
}
}

```